

CS285 LECTURE 10 • ZERO-TO-HERO FIELD MANUAL

Proximal Policy Optimization (PPO) & Trust Regions

From Scratch to Mastery: Understanding Why Reinforcement Learning Collapses, How Clipping Saves It, and Why PPO Rules Modern Robotics

Instructor: Prof. Sergey Levine (UC Berkeley RAIL Lab) **Focus:** Policy Collapse, Kakade-Langford Bounds, TRPO, & The PPO Clipped Objective

1. What is Policy Collapse? (The Student Burning Their Books)

In supervised machine learning (like image classification), if you choose a learning rate that is too high, the loss spikes on epoch 12. You don't panic. On epoch 13, the network sees more normal images, and the loss recovers. **In Reinforcement Learning, an oversized learning rate causes instant, permanent death.** Why?

☀ THE CATASTROPHIC RL DEATH SPIRAL

In RL, **the policy collects its own training data!**

- Imagine a student studying for an important medical exam.
- On Friday, the student tries a slightly unusual study method. They take a practice test on Saturday and get a mediocre score.
- Instead of making a minor 2% adjustment, the student panics, **burns all their textbooks, deletes their memory, and forgets how to read!**
- Because they forgot how to read, every practice test they take next week scores a zero. Because every test is a zero, they can never learn anything new.

This is **Policy Collapse**: if an aggressive gradient update moves network weights into an unstable region, the robot starts flailing wildly. All 1,024 parallel environments in Isaac Sim generate pure garbage trajectories. Because the new data is 100% garbage, the next gradient update makes the policy even worse. Training dies permanently!



2. The Solution: PPO & The Bowling Bumpers

🎳 THE BOWLING ALLEY BUMPERS ANALOGY

When children play bowling, the alley puts up **inflatable bumpers** along the gutters. No matter how wildly the child throws the ball, the ball bounces off the bumper, stays in the lane, and hits pins.

Proximal Policy Optimization (PPO) puts bumpers on neural network updates! It says: *"No matter how enthusiastic the gradient is about an action, the new policy is NEVER allowed to deviate by more than 20% ($\epsilon = 0.2$) from the old policy that collected the data."*

2.1 The PPO Clipped Surrogate Objective (Parameter Anatomy)

First, define the **Probability Ratio (The Speedometer)**:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

Then, the **PPO Clipped Objective (The Bumpers)**:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

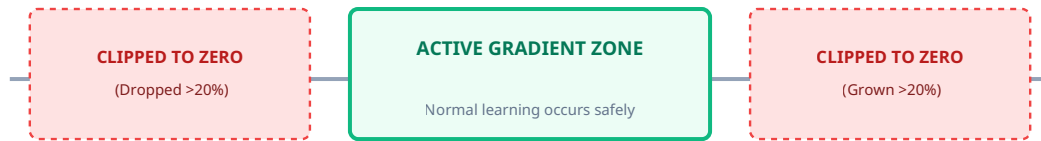
Parameter	Formal Name	Plain English Meaning	Recommended Value
$r_t(\theta)$	Probability Ratio	The Speedometer: How much more or less likely is action a_t under the new policy compared to the old policy?	$r = 1.0$ at start of batch; rises or falls during training.
ϵ	Clipping Threshold	The Speed Limit: The maximum allowed percentage change in action probability.	$\epsilon = 0.2$ (Allows $\pm 20\%$ change: $[0.8, 1.2]$).
$\hat{A}_t$	Advantage Score	Was this robot motion better ($\hat{A} > 0$) or worse ($\hat{A} < 0$) than average?	Computed via $\text{GAE}(\lambda = 0.95)$.
$\min(\dots)$	Pessimistic Bound	Forces the algorithm to take the more conservative, pessimistic estimate so it never over-promises.	Guarantees monotonic policy improvement.

PLAIN ENGLISH TRANSLATION OF THE PPO LOSS

"If an action was good ($\hat{A} > 0$), make it more likely—but once you have increased its probability by 20% ($r > 1.2$), **STOP** pushing and ignore further gradients! If an action was bad ($\hat{A} < 0$), decrease it—but once it dropped by 20% ($r < 0.8$), **STOP** punishing it!"

3. The 4-Quadrant Matrix: How PPO Responds to Every Situation

Every single timestep in an Isaac Lab batch falls into one of four clear quadrants:



Scenario	Advantage ($\hat{A}$)	Ratio Condition (r_t)	What PPO Does (Plain English)
Quadrant 1	$\hat{A} > 0$ (Good cut)	$r_t \in [1.0, 1.2]$ (Safe increase)	Normal Positive Gradient: Robot keeps increasing the probability of this successful motion.
Quadrant 2	$\hat{A} > 0$ (Good cut)	$r_t > 1.2$ (Exceeded 20%)	Clipped to 0 Gradient: Stop pushing! You already made this action 20% more likely. Don't over-commit.
Quadrant 3	$\hat{A} < 0$ (Crushed fruit)	$r_t \in [0.8, 1.0]$ (Safe decrease)	Normal Negative Gradient: Suppresses the probability of this crushing mistake.
Quadrant 4	$\hat{A} < 0$ (Crushed fruit)	$r_t < 0.8$ (Dropped >20%)	Clipped to 0 Gradient: Stop punishing! The probability has already dropped significantly. Leave it alone.

4. Why PPO Rules GPU Robotics (NVIDIA Isaac Lab)

⚡ THE 10X SPEEDUP SECRET: MULTI-EPOCH DATA REUSE

In vanilla policy gradients (Lecture 5), you simulate 1,024 parallel robots on GPU, take **ONE single gradient step**, and you **MUST** throw away all data immediately. Why? Because the data is now off-policy! Generating 10 million transitions to take 1,000 steps wastes 99% of your GPU power.

Why PPO dominates Isaac Lab: Because PPO's clipping mechanism guarantees safety, you can take the exact same batch of GPU simulation data and train on it for **4 to 8 epochs** across mini-batches! You extract $8\times$ more learning out of every single simulation frame.

5. Concrete Numerical Walkthrough: 4 Cases with Real Numbers

Let's calculate PPO's objective value across four concrete actions with clipping threshold $\epsilon = 0.20$:

Case	Advantage $\hat{A}$	Ratio $r_t(\theta)$	Unclipped Term $r_t \hat{A}_t$	Clipped Term $\text{clip}(r) \hat{A}_t$	Final Loss
1. Moderate Win	+5.0 (Good)	1.10 (+10%)	$1.10 \times 5.0 = 5.50$	$1.10 \times 5.0 = 5.50$	$\min(5.5, 5.5) = \mathbf{5.50}$
2. Runaway Win	+5.0 (Good)	1.45 (+45%)	$1.45 \times 5.0 = 7.25$	$1.20 \times 5.0 = 6.00$	$\min(7.25, 6.0) = \mathbf{6.00}$ (Clipped!)
3. Moderate Loss	-4.0 (Bad)	0.90 (-10%)	$0.90 \times (-4.0) = -3.60$	$0.90 \times (-4.0) = -3.60$	$\min(-3.6, -3.6) = \mathbf{-3.60}$
4. Massive Loss	-4.0 (Bad)	0.65 (-35%)	$0.65 \times (-4.0) = -2.60$	$0.80 \times (-4.0) = -3.20$	$\min(-2.6, -3.2) = \mathbf{-3.20}$ (Clipped!)

6. Complete PyTorch PPO Loss Implementation

🐍 COMPLETE PYTORCH PPO CLIPPED LOSS WITH ENTROPY BONUS

```
import torch
import torch.nn as nn

def compute_ppo_loss(log_probs, old_log_probs, advantages, entropy, clip_eps=0.2, c_entropy=0.01):
    # 1. Compute Probability Ratio:  $r(\theta) = \exp(\log\_prob - \text{old\_log\_prob})$ 
    ratio = torch.exp(log_probs - old_log_probs)

    # 2. Unclipped and Clipped Surrogate Objectives
    surr1 = ratio * advantages
    surr2 = torch.clamp(ratio, 1.0 - clip_eps, 1.0 + clip_eps) * advantages

    # 3. PPO Policy Loss: Take minimum (pessimistic bound) and negate for optimizer
    policy_loss = -torch.min(surr1, surr2).mean()

    # 4. Entropy Bonus: Encourages exploration of dynamic impedance values
    entropy_loss = -c_entropy * entropy.mean()

    total_loss = policy_loss + entropy_loss
    return total_loss, policy_loss.item(), ratio.mean().item()
```

7. Practitioner's Failure Modes & Debugging Checklist

Failure Mode	The Hidden Symptom	How to Fix It
1. KL Divergence Explosion	Policy changes too rapidly despite clipping. Approximate KL divergence $KL \approx \text{mean}((\text{ratio} - 1) - \log(\text{ratio}))$ spikes above 0.05.	Use an Adaptive Learning Rate Scheduler (like <code>KLAdaptiveLR</code> in <code>SkRL</code>): if $KL > 0.02$, lower learning rate; if $KL < 0.005$, increase it.
2. Too Many Epochs Per Batch	Setting epochs: 20 causes the policy to overfit to the batch. The ratio blows past $[0.8, 1.2]$ and 90% of transitions get clipped to zero gradients.	In Isaac Lab, set epochs: 4 or 5 . More epochs provide diminishing returns.
3. Clipping the Critic Target	Clipping the Value function loss (V -clip) was popular in 2018, but recent ablation studies show it actively harms performance in continuous robotics.	Leave the Critic unclipped: train $V(s)$ with standard MSE loss.

CS285 Lecture 10 Zero-to-Hero Guide • DEX-ROB Lab (Tianjin University) • Prof. Shan An